COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Jul 9, 2026	**DATE SUBMITTED:**	Jul 22, 2026
NAME:	Ellaiza C. Vizcayno	**INSTRUCTOR:**	Mr. Jimlord Quejado

ACTIVITY NO. 1.1 **Basic C++ Programming**

1. PROCEDURE OUTPUT

Sections	**Answer**
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	none

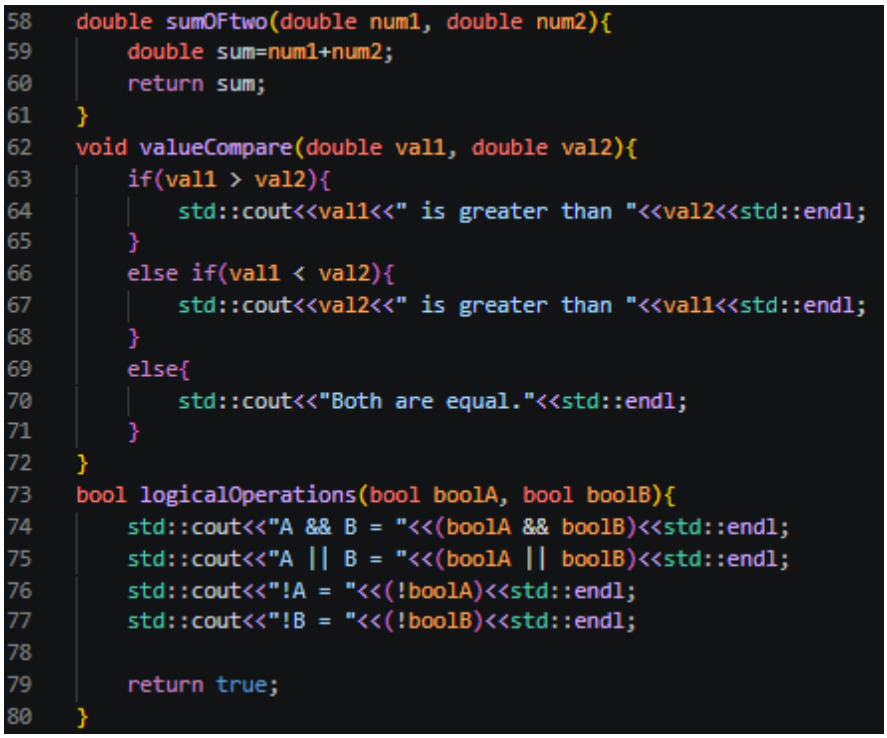
Class Declaration and Method Definition Section	`double sumOFtwo(double num1, double num2);`
	`void valueCompare(double val1, double val2);`
	`bool logicalOperations(bool boolA, bool boolB);`

Main Function

```cpp
int main(){
    int userInput;
    double num1, num2;
    bool boolA, boolB;

    do{
        std::cout<<"=== >OPERATIONS< ===\n"
                   "0) Exit\n"
                   "1) Sum of 2 Numbers\n"
                   "2) Compare 2 Values\n"
                   "3) Boolean Logical Operations\n\n"
                   "Input number from menu: ";
        std::cin>>userInput;
        switch(userInput){
            case 0:
                break;
            case 1:
                std::cout<<"\n=== >SUM OF 2 NUMBERS< ===\n";
                std::cout<<"Enter number for value 1: ";
                std::cin>>num1;
                std::cout<<"Enter number for value 2: ";
                std::cin>>num2;
                std::cout<<num1<<" + "<<num2<<sumOFtwo(num1, num2)<<std::endl;
                std::cout<<"\n==================\n";
                break;
            case 2:
                std::cout<<"\n=== >COMPARE 2 VALUES< ===\n";
                std::cout<<"Enter number for value 1: ";
                std::cin>>num1;
                std::cout<<"Enter number for value 2: ";
                std::cin>>num2;
                valueCompare(num1, num2);
                std::cout<<"\n==================\n";
                break;
            case 3:
                std::cout<<"\n=== >BOOLEAN LOGICAL OPERATIONS< ===\n";
                std::cout<<"Enter 1st Boolean Value(1 = true, 0 = false): ";
                std::cin>>boolA;
                std::cout<<"Enter 2nd Boolean Value(1 = true, 0 = false): ";
                std::cin>>boolB;
                logicalOperations(boolA, boolB);
                std::cout<<"\n==================\n";
                break;
            default:
                std::cout<<"\nINVALID\n"<<std::endl;
        }
    }while(userInput != 0);

    return 0;
}
```

Method Definition	

```
58  double sumOFtwo(double num1, double num2){
59      double sum=num1+num2;
60      return sum;
61  }
62  void valueCompare(double val1, double val2){
63      if(val1 > val2){
64          std::cout<<val1<<" is greater than "<<val2<<std::endl;
65      }
66      else if(val1 < val2){
67          std::cout<<val2<<" is greater than "<<val1<<std::endl;
68      }
69      else{
70          std::cout<<"Both are equal."<<std::endl;
71      }
72  }
73  bool logicalOperations(bool boolA, bool boolB){
74      std::cout<<"A && B = "<<(boolA && boolB)<<std::endl;
75      std::cout<<"A || B = "<<(boolA || boolB)<<std::endl;
76      std::cout<<"!A = "<<(!boolA)<<std::endl;
77      std::cout<<"!B = "<<(!boolB)<<std::endl;
78
79      return true;
80  }
```

Table 1-1. C++ Structure Code for Answer


For this part of the activity, we're learning about encapsulation which is locking the data, unable to be modified by code outside the class. The way to encapsulate the data would be using private and this data would only be accessible by all member functions of the class. Then at the bottom would be the method definition where you define the functions declared. These functions are able to access the private variables. When you input the parameters, the parameters would be assigned to the private variables; angleA, angleB, angleC, and totalAngle. The constructor definition initializes the object that is going to be used for the method definition after being created. Then the method definition sets the angles according to the data given and the totalAngle is computed. After the triangle will be validated according to the totalAngle.

```
The shape is a valid triangle
```

Table 1-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Create a C++ program to swap the two numbers in different variables.
Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.
Create a C++ program that has a function that will calculate the distance between two points.

```cpp
#include <iostream>
#include <cmath>

void swapNum(int& a, int& b);
float convertKtoF(float k);
float distanceBetTwo(float x1, float y1, float x2, float y2);

int main(){
    int userInput;
    int num1, num2;
    float kelvin;
    float x1, y1, x2, y2;

    do{
        std::cout<<"=== >OPERATIONS< ===\n"
                   "0) Exit\n"
                   "1) Swap 2 Numbers\n"
                   "2) Convert Kelvin to Fahrenheit\n"
                   "3) Distance Between 2 Points\n\n"
                   "Input number from menu: ";
        std::cin>>userInput;
        switch(userInput){
            case 0:
                break;
            case 1:
                std::cout<<"\n=== >SWAP 2 NUMBERS< ===\n";
                std::cout<<"Enter first number: ";
                std::cin>>num1;
                std::cout<<"Enter second number: ";
                std::cin>>num2;
                std::cout<<"Before swap: "<<num1<<" "<<num2<<std::endl;
                swapNum(num1, num2);
                std::cout<<"After swap: "<<num1<<" "<<num2<<std::endl;
                std::cout<<"\n==================\n";
                break;
            case 2:
                std::cout<<"\n=== >KELVIN TO FAHRENHEIT< ===\n";
                std::cout<<"Enter temperature in Kelvin: ";
                std::cin>>kelvin;
                std::cout<<kelvin<<" K = "<<convertKtoF(kelvin)<<" F"<<std::endl;
                std::cout<<"\n==================\n";
                break;
```

```
                case 3:
                    std::cout<<"\n=== >DISTANCE BETWEEN 2 POINTS< ===\n";
                    std::cout<<"Enter x1: ";
                    std::cin>>x1;
                    std::cout<<"Enter y1: ";
                    std::cin>>y1;
                    std::cout<<"Enter x2: ";
                    std::cin>>x2;
                    std::cout<<"Enter y2: ";
                    std::cin>>y2;
                    std::cout<<"Distance = "<<distanceBetTwo(x1, y1, x2, y2)<<std::endl;
                    std::cout<<"\n==================\n";
                    break;
                default:
                    std::cout<<"\nINVALID\n"<<std::endl;
            }
        }while(userInput!=0);

        return 0;
    }
    void swapNum(int& a, int& b){
        int temp = a;
        a = b;
        b = temp;
    }
    float convertKtoF(float k){
        float f = (k - 273.15)*(1.8)+32;
        return f;
    }
    float distanceBetTwo(float x1, float y1, float x2, float y2){
        float distance = sqrt(pow(x2-x1, 2) + pow(y2-y1, 2));
        return distance;
    }
```

```
=== >OPERATIONS< ===
0) Exit
1) Swap 2 Numbers
2) Convert Kelvin to Fahrenheit
3) Distance Between 2 Points

Input number from menu: 4

INVALID
```

```
=== >OPERATIONS< ===
0) Exit
1) Swap 2 Numbers
2) Convert Kelvin to Fahrenheit
3) Distance Between 2 Points

Input number from menu: 1

=== >SWAP 2 NUMBERS< ===
Enter first number: 57
Enter second number: 38
Before swap: 57 38
After swap: 38 57
```

```
=== >OPERATIONS< ===
0) Exit
1) Swap 2 Numbers
2) Convert Kelvin to Fahrenheit
3) Distance Between 2 Points

Input number from menu: 2

=== >KELVIN TO FAHRENHEIT< ===
Enter temperature in Kelvin: 24
24 K = -416.47 F
```

```
=== >OPERATIONS< ===
0) Exit
1) Swap 2 Numbers
2) Convert Kelvin to Fahrenheit
3) Distance Between 2 Points

Input number from menu: 3

=== >DISTANCE BETWEEN 2 POINTS< ===
Enter x1: 3
Enter y1: 6
Enter x2: 2
Enter y2: 9
Distance = 3.16228
```

ANALYSIS

The purpose of these programs is to demonstrate how functions can be used to perform different tasks while keeping the code organized and easier to manage. Each function has a specific responsibility: swapping two numbers, converting a temperature from Kelvin to Fahrenheit, and calculating the distance between two points. Since each task requires different data and produces different results, separate functions make the program more structured and easier to understand.

The program applies modular programming by dividing each operation into its own function. This avoids repeating the same set of statements whenever the operation is needed and makes the code easier to read, debug, and maintain. It also becomes easier to work on the program since each function has a clear purpose.

Each function demonstrates a different way of handling parameters and return values. The swapNum() function uses pass-by-reference because the values of the variables need to be modified directly. If pass-by-value were used, the changes would only exist inside the function and the original variables would remain unchanged. On the other hand, convertKtoF() and distanceBetTwo() only perform calculations and return the computed result, which is why they return a float instead of modifying existing variables.

Separating each operation into its own function allows the program to handle different tasks without placing all statements in one section. Each function focuses on a specific responsibility, which helps maintain a clear flow of operations while preventing unnecessary repetition. This makes functions a useful approach when solving programming problems that involve multiple processes.

Modify ILO B

```cpp
#include <iostream>

//naming convention for class name: PascalCase
class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
        double base, height;
        double side1, side2, side3;
    public:
        //constructor
        Triangle(double A, double B, double C, double Ba,
             double H, double S1, double S2, double S3);
        void setAngles(double A, double B, double C);
        void setDimensions(double Ba, double H, double S1,
                           double S2, double S3);
        double findArea();
        double findPerimeter();
        void triangleTypeAngles();
        void triangleTypeSides();
        //triangle validation
        const bool validateTriangle();
};
int main(){

    Triangle triangle(60, 60, 60, 10, 8, 10, 10, 10);
```

```cpp

    std::cout<<"Area: "<<triangle.findArea()<<std::endl;
    std::cout<<"Perimeter: "<<triangle.findPerimeter()<<std::endl;

    std::cout<<"Triangle Type by Angles: ";
    triangle.triangleTypeAngles();

    std::cout<<"Triangle Triangle by Sides: ";
    triangle.triangleTypeSides();

    return 0;
}
Triangle::Triangle(double A, double B, double C, double Ba,
                   double H, double S1, double S2, double S3){
    angleA=A;
    angleB=B;
    angleC=C;
    totalAngle=A+B+C;
    base=Ba;
    height=H;
    side1=S1;
    side2=S2;
    side3=S3;
}
void Triangle::setAngles(double A, double B, double C){
```

```cpp
void Triangle::setAngles(double A, double B, double C){
    angleA=A;
    angleB=B;
    angleC=C;
    totalAngle=A+B+C;
}
void Triangle::setDimensions(double Ba, double H, double S1,
                             double S2, double S3){
    base=Ba;
    height=H;
    side1=S1;
    side2=S2;
    side3=S3;
}
double Triangle::findArea(){
    return (base * height) / 2;
}
double Triangle::findPerimeter(){
    return side1 + side2 + side3;
}
void Triangle::triangleTypeAngles(){
    if(angleA < 90 && angleB < 90 && angleC < 90){
        std::cout<<"Acute"<<std::endl;
    }else if(angleA == 90 || angleB == 90 || angleC == 90){
        std::cout<<"Right"<<std::endl;
    }else if(angleA > 90 || angleB > 90 || angleC > 90){
        std::cout<<"Obtuse"<<std::endl;
    }else{
        std::cout<<"Invalid"<<std::endl;
    }
}
void Triangle::triangleTypeSides(){
    if (validateTriangle()){
        if(side1 == side2 && side1 == side3){
            std::cout<<"Equilateral"<<std::endl;
        }else if((side1 == side2 && side2 != side3)||
                 (side2 == side3 && side3 != side1) ||
                 (side3 == side1 && side1 != side2)){
            std::cout<<"Isosceles"<<std::endl;
        }else if(side1 != side2 && side1 != side3 &&
                 side2 != side3){
            std::cout<<"Scalene"<<std::endl;
        }else{
        std::cout<<"Invalid"<<std::endl;
    }
    }else{
        std::cout<<"Invalid"<<std::endl;
    }
}
const bool Triangle::validateTriangle(){
    return(totalAngle<=180);
}
```

```
Area: 40
Perimeter: 30
Triangle Type by Angles: Acute
Triangle Triangle by Sides: Equilateral
```

ANALYSIS

The purpose of this program is to demonstrate object-oriented programming by representing a triangle as an object. Instead of storing the triangle's data in separate variables, the program groups its angles, side lengths, base, height, and related operations into a single Triangle class. Since all of these belong to the same object, organizing them in one class makes the program easier to understand and manage.

The constructor is used to initialize the triangle as soon as the object is created. Instead of declaring the object first and assigning each attribute one by one, all the required values are provided when the object is instantiated. Setter functions are also included to allow the object's data to be updated later without creating another Triangle object whenever changes are needed.

Each member function has a specific responsibility. The findArea() and findPerimeter() functions calculate and return numerical values, which is why they return a double. Meanwhile, triangleTypeAngles() and triangleTypeSides() determine the triangle type using different characteristics. Separating these functions keeps the code more organized since angle classification and side classification follow different conditions.

The validateTriangle() function checks whether the given angles form a valid triangle before the program classifies it. This prevents the program from identifying a triangle type when the given measurements do not represent a valid triangle, improving the reliability of the program.

Through the use of the Triangle class, different triangle objects can be created using the same structure while still having their own assigned values. Since each object contains its own characteristics, the program can perform calculations and classifications based on the given measurements without needing separate implementations for each triangle.

CONCLUDING STATEMENT / FEEDBACK

To be honest, I think I did well in this activity. I was able to accomplish the given tasks and understand the concepts being applied. One of the areas I still struggle with is pointers, as I still get confused with the concept from time to time. This shows that I need to spend more time familiarizing myself with how pointers work and how they are used in different situations.

Another area I need to improve on is knowing when additional features or functions are necessary. While adding more functionality can help improve a program, I realized that not every laboratory activity needs to be approached like a final project. I tend to add features such as CRUD operations even when they are not required, which can make the program more complex than needed. Moving forward, I want to improve my ability to focus on the given requirements first and add improvements only when they serve a clear purpose.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **When confused with a concept, I ask it various questions until I can understand it. I avoid asking for the direct answer and instead give me clues so that I could still think for myself.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.